AMAI: Algebraic Modelling of Distributed AI Computations for Symbolic Performance Analysis

Rubén Coll Sánchez
Sorbonne University &
Huawei Technologies France SASU
Boulogne-Billancourt, France
rubencollsanchez@gmail.com

Thibaut Tachon
Huawei Technologies France SASU
Boulogne-Billancourt, France
thibaut.tachon@huawei.com

Pierre Leca
Huawei Technologies France SASU
Boulogne-Billancourt, France
pierre.leca@huawei.com

Teng Su
Huawei Technologies Co. Ltd
Hangzhou, China
suteng@huawei.com

Chong Li
Huawei Technologies France SASU
Boulogne-Billancourt, France
ch.l@huawei.com

***Abstract*—Developing large-scale AI systems requires extensive on-machine tuning: engineers evaluate hundreds of parallel configurations by trial and error, at considerable cost in accelerator rental and compute time. Existing performance models are either tied to specific frameworks or too coarse for modern distributed workloads. We present AMAI, a performance analysis framework featuring (i) a declarative domain-specific language for describing distributed deep-learning computations, and (ii) compositional cost semantics that map programs into symbolic models of time and per-device memory. Models are parameterized by hardware constants and structural variables that define the parallelism configuration. As a result, descriptions can be analyzed at any scale and reused across machines. Predictions stay within 3 percent of measured step time and peak memory across batch-size and data-parallel scaling when analyzing the training of Qwen3.5 0.8B on a multi-node NPU cluster.**

***Index Terms*—Performance analysis, Computational modeling, Parallel programming, Deep learning, Software performance.**

I. Introduction

Over the last few decades, AI models have gained traction as effective tools for solving complex problems. More specifically, deep-learning models such as LLMs have become particularly interesting, since their design scale can be easily increased to learn from enormous amounts of data. As a result, they are in high demand, and there is an ever-growing interest in training and using them [1]. Notable stories of success are the LLaMa series from Meta, Anthropic's Claude, Gemini from Google and OpenAI's ChatGPT. In the next years, many public and private institutions are bound to offer solutions based on machine intelligence and automated agentic systems that perform thousands of tasks [1]–[3].

At the same time, AI-based systems require great computing power. To meet the unprecedented demand for AI services, computer architectures and data centers have grown exponentially in size and performance [4]. More powerful, specialized hardware is designed and manufactured every year to match the industry requirements. A cluster for AI computing can hold hundreds of specialized accelerators, interconnected through high-bandwidth links to form a large distributed computing network. In order to exploit them to the fullest, engineers need to modify and parallelize the deep-learning algorithms that run on them, so that work is split efficiently across different computing units [5]. Many programming frameworks help to ease the task of distributed programming, by transforming the high-level mathematical description of AI workloads into high-performance low-level code automatically [6]–[9]. Even so, depending on the algorithm in question, the span of applicable parallel configurations can be huge [5].

Currently, only a few private companies can afford to invest in their own infrastructure. The vast majority of organizations rely instead on Infrastructure-as-a-Service (IaaS), and rent the computing power they require. This business model has been successful for years: providers orchestrate services like databases or remote storage on the cloud, and consumers access them transparently [10]. However, the costs incurred in infrastructure during development of novel, innovative AI solutions are far greater than those of developing classic software systems. Building, tuning and testing AI-powered systems, like inference engines or swarms of agents, includes analyzing the resource requirements of the system to verify that the overall performance of the underlying algorithms is high enough. It takes time to perform trial and error on hundreds of configurations, with the consequent costs in machine rental and compute time. Current state-of-the-art AI models already require big compute clusters to be used, and it is expected that the price of allocating resources keeps increasing. Some service providers offer a solution: dedicated engineering teams that tune and optimize solutions deployed on their machines. This option comes at the cost of sharing important strategic information, internal design details about the system, confidential parameters and other compromising assets. While this alternative can help innovative companies in the short term, it also exposes them to their own service providers, which could try to replicate their services or adopt a position against them if their business interests conflict.

SC26 Workshops, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-1221-5/26/$31.00 ©2026 IEEE

Several methods for performance analysis and optimization address this issue. Many of them interpret certain subcomponents of AI-based systems as parameterized instances of a general pattern, and construct a search space over a certain metric of interest [5], [11]. Some others provide an abstract view of the hardware, along with rules and equations that model the performance of the algorithms that run on them [12]–[14]. Both approaches have proved effective in their use cases, yet no method solves specifically the problem of modelling the performance of deep-learning systems while remaining general enough to be applied in a diversity of cases.

Our aim is to provide a method for analyzing the performance of deep-learning computations that minimizes the impact of on-machine validation. We believe that such a method should allow flexibility by presenting three properties of interest:

- The first one is compositionality. The meaning and behaviour of a complex design should be completely determined by the combination of its components. This property is essential to allow analysis at any scale; without it, the method's complexity grows with the size of the input.
- The second one is operability. Analyzed systems and solutions may be heterogeneous: some models may require structural information, and some others may need empirical data. The user should be able to combine different representations of arbitrary precision, without constraining the design of the whole.
- The third one is parametricity. Once the representation of a computation has been created, it should be possible to concretize it over a broad range of computing models. This property ensures generality and reusability over time.

To the best of our knowledge, no existing work targeting our specific problem presents all three of them. Our main contribution is AMAI, a software system conceived to analyze and predict the performance of parallel and distributed computations for AI/ML workloads. As an end-to-end tool, it can deduce costs systematically by combining high-level algebraic descriptions of programs, and hardware parameters from the machines that execute them. The usage workflow of AMAI is depicted in Figure 1. The user specifies a program using a lightweight domain-specific language, and the system automatically extracts a set of symbolic cost models for it. Cost models' parameters are related to the compute systems that execute the workload, such as compute latencies, bandwidths, or memory sizes. They can be traced down, extracted and fitted with known or empirically measured values. Once the parameter values are obtained, it is possible to evaluate the models using them, and obtain final resource requirements.

The outline of the paper is the following: Section II introduces background on distributed deep-learning computations and performance cost models. In Section III we present our domain-specific modelling language as a formal tool to abstract the properties of deep-learning computations. In Section IV we develop its embedded cost model framework with the mapping that parses AMAI representations into parameterized cost model instances. Finally, we validate the method as a performance analysis system through experimental evaluation in Section V, and test its capabilities. Section VI completes our work with a review of the related work done in the field, and Section VII concludes.

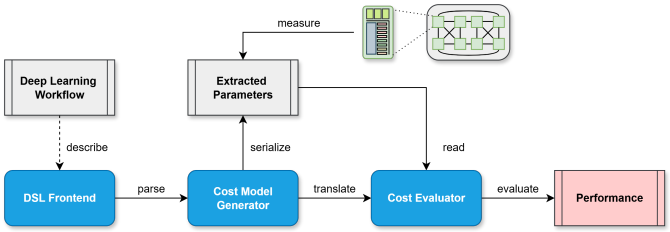


Fig. 1. Global view of AMAI's structure

II. Background

A. Distributed deep learning

Deep neural networks (DNNs) are at the core of the vast majority of AI models. They express complex functions by chaining blocks of linear and non-linear operators that act on tensor inputs and parameters. Tensors are represented with a multi-dimensional array of numerical data; they generalize real numbers, vectors, and matrices. There are several training schemes for deep neural networks, depending on the model's objective and the learning task selected [15], [16]. In many of them, training consists in iteratively updating the parameters (or *weights*) of the network using first-order gradients. State-of-the-art methods for supervised training of foundation models usually divide the training data in batches and process them in steps. Each training step consists of calculating gradients with respect to the weights and then updating the weights with them. The weight update rule usually includes an optimizer that maintains running statistics of past gradients [17], [18].

Programming the training algorithm for a DNN is a non-trivial task. It has to be both mathematically correct and achieve high performance to be able to deal with large training datasets. State-of-the-art AI and scientific computing frameworks approach the issue with algorithms that automatically differentiate DNNs to calculate sound gradients [6], [19], [20] and optimize the computations for minimum time and memory consumption. With them, programmers can prototype, test, and deploy models quickly, while the compilers underneath manage the intermediate computational representations and output optimized, highly efficient code for different hardware architectures.

As a drawback, classic techniques for parallel computing cannot be applied directly, since they need to be integrated with the algorithms that transform the code. The solution is to implement these techniques directly in the framework and allow the programmer to use them with exposed configuration parameters. One of them is explicit *data parallelism*: when training a model, a user can specify to divide the input data

across *DP* workers and process it in parallel. The system will replicate the model across computing units and introduce data communication directives (namely the *AllReduce* collective, in this case) to gather all the gradients before updating the weights, so that the training scheme remains mathematically correct. Another one is *model parallelism*: the model's operators and parameters are sharded uniformly across *MP* workers that will compute partial functions and share results. The system will calculate partitions, generate partial computing graphs and introduce communication collectives (*AllGather*, *ReduceScatter* among others) to redistribute the data as needed for each sharded operator. Users can choose among dozens of more advanced techniques in the latest frameworks [7], [8], [21], each one introducing a new dimension in the configuration space of parallel strategies. Predicting the performance gain of a combination of parallel techniques is a hard task that requires expert knowledge of the framework's internal implementation; in the worst case, each configuration must be profiled and analyzed individually.

B. Performance models for parallel computation

Reasoning about the performance of parallel programs without executing them is the founding concern of bridging models of parallel computation, which act as a connection between portable, machine-agnostic parallel programming and hardware-aware performance prediction. The idea is to abstract the machines as a small set of parameters and represent their behaviour with them. After implementing algorithms on these machines, it is possible to reason about the time and the memory they consume. The more closely the abstract machine resembles the real one, the more precise are the estimations. Some prominent families of bridging models in this line of work are the BSP family of machines [13] and the LogP-based models [14], [22].

The bulk-synchronous parallel model (BSP) [13] represents a parallel machine with three parameters: the number of processors (P), the communication gap (g), and the barrier synchronization latency (L). Every computation is structured as a sequence of supersteps, each consisting of local computation, message exchange, and global barrier. Extensions of BSP, such as Multi-BSP [23] and SGL [24] decompose the machine hierarchically to capture submachines and memory hierarchies.

The LogP bridging model characterizes asynchronous point-to-point communication through four parameters: latency (L), per-message overhead (o), minimum inter-message gap (g), and the number of processors (P). This finer granularity allows it to model irregular, unstructured communication patterns that BSP's superstep abstraction cannot capture. LogGP is a direct extension of LogP that adds a fifth parameter, G, the gap per byte for long messages, and corrects LogP's implicit assumption that all messages are short and of fixed size.

Another notable mention is the α-β model, first introduced in [25]. It focuses on characterizing communications between processes. The model expresses the time to send a message whose size is n bytes as the sum of a fixed startup latency (α) and an inverse bandwidth term (β), in the form of $t(n) = \alpha+\beta\cdot n$. Its simplicity made it the de facto model for analyzing collective communication in subsequent work on distributed memory computations and architectures [26], [27].

These performance models assume that the algorithm being analyzed closely matches the program executed on the target machine. This assumption no longer holds for modern deep-learning workloads, which undergo several transformations (automatic differentiation, layout transformations, partitioning) as frameworks compile their initial specification. Consequently, the computation and communication patterns observed at execution time differ substantially from those expressed in the original program. The ideas of classical performance models cannot be applied without translating the final workload of interest into a representation that can be used to judge and reason with. For lack of a dedicated representation of this kind, the choice is usually to use similar intermediate representations that serve other purposes within the framework stack, at the cost of losing design information.

III. Modelling language

In this section, we introduce the domain-specific language of AMAI, conceived to specify and encode cost models of performance. As such, it is not interpreted or compiled to executable code. With it, users can materialize their hypothesis and assumptions about the behaviour of the system they intend to analyze. In other words, it simplifies the modeling process by shifting the focus from *what* to compute to *how* to compute.

Every expression in AMAI's DSL represents a deep-learning computation, and each term is a combination of simpler subterms. The compositionality of the system is a direct consequence. We implemented it in OCaml, a functional programming language that offers higher-order programming facilities for minimal design overhead. This choice decouples AMAI from any programming framework, and eases its integration as part of a workflow for performance modeling: operability is maximized.

We draw inspiration from the arrow pattern, an extensively used functional programming pattern that encapsulates functions and makes their composition in sequence and in parallel explicit [28]. We extend the system with three additional combinators that describe structured parallel computations as tensor operations, which are widely used for AI development. The complete description of AMAI is given in Figure 2 through an interface that shows how to construct AMAI terms.

```
type shape = int list
type 'a tensor = ('a list) * shape

type ('a,'b) t

val atom   : string -> ('a -> 'b) -> ('a,'b) t
val (>>>)  : ('a,'b) t -> ('b,'c) t -> ('a,'c) t
val (|||)  : ('a,'b) t -> ('c,'d) t -> ('a*'c, 'b*'d) t

val par    : shape -> ('a,'b) t -> ('a tensor,'b tensor) t
val extend : shape -> ('a,'b) t -> ('a,'b tensor) t
val reduce : shape -> ('a*'a,'a) t -> ('a tensor,'a) t
```

Fig. 2. The syntax of AMAI's DSL as a module interface.

The basic constructor of computations is *atom*: it encapsulates an arbitrary function retaining the input and output types, and takes a label that identifies it globally. Its purpose is to serve as the smallest building block for describing computations. Any details about the function it encapsulates are meant to be forgotten in the model. The constructor of sequences ($\ggg$) can be seen as an operator that composes functions in postfix notation. With it, it is possible to chain two compatible computations for which the output of the former is the same type as the input of the latter. The constructor of parallel computations ($|||$) joins two computations as one, and combines inputs and outputs with a type product.

Structured parallelism is achieved with the *par* directive. It takes a tensor shape and a computation, and describes the application of the computation across all elements of a tensor with the given shape. The *extend* directive is similar: with the same arguments, it replicates the computation, creating different copies that apply over the same input. As a dual of these two, *reduce* contracts tensors of a specific shape using a computation as a binary operator, assumed associative. The typing of AMAI programs does not check tensor shapes, deliberately. It performs a lighter check, implicitly comparing tensor types, since we do not require strong type checking for the purposes of our contribution.

As an example of usage, we show how to describe some common components within deep-learning models in Figure 3. These constructions are incremental, and parallelism is added step by step. Note how the system is flexible enough to admit placeholders where some of the underlying details might be unknown. This is the case for windowing inside convolutions: the accessing of different subtensors to apply the kernel is hidden in the *window* function. It is an example of the operability of our method, and lets the user bypass otherwise impossible constraints, such as detailing the memory hierarchy when its details are unknown. We use the same mechanism for loading too, in this case, because we choose to bypass the modelling of memory consistency.

```
(* Create new combinators *)
let id = atom "id" (fun x -> x)
let dup = atom "dup" (fun x -> (x,x))
let first c = c ||| id

(* Play with basic functions *)
let add = atom "+" (fun (x,y) -> x +. y)
let mul = atom "*" (fun (x,y) -> x *. y)
let square = dup >>> mul

(* Describe a linear layer *)
let load = atom "load" (fun _ -> init ())
let dot k =
  par [k] (dup >>> (load ||| id) >>> mul)
  >>> reduce [k] add
let linear k n = extend [n] (dot k)

(* Describe a convolutional layer *)
let window = atom "window" (fun x -> x)
let kernel k1 k2 =
  extend  [k1;k2] (dup >>> (load ||| window) >>> mul)
  >>> reduce  [k1;k2] add
let conv2d n m k1 k2 = par [n;m] (kernel k1 k2)
```

Fig. 3. Some examples using AMAI operations to encode components of deep learning computations

$$\frac{\Gamma \vdash s, s' : \mathit{string} \quad \Gamma \vdash f, g : \texttt{a -> b} \quad s = s'}{\Gamma \vdash \mathit{atom}\ s\ f \equiv_\Phi \mathit{atom}\ s'\ g}$$

$$\frac{\Gamma \vdash p, p' : \texttt{(a,b) t} \quad \Gamma \vdash q, q' : \texttt{(b,c) t} \quad p \equiv_\Phi p' \quad q \equiv_\Phi q'}{\Gamma \vdash p \ggg q \equiv_\Phi p' \ggg q'}$$

$$\frac{\Gamma \vdash p, p' : \texttt{(a,b) t} \quad \Gamma \vdash q, q' : \texttt{(c,d) t} \quad p \equiv_\Phi p' \quad q \equiv_\Phi q'}{\Gamma \vdash p \,|||\, q \equiv_\Phi p' ||| \, q'}$$

$$\frac{\Gamma \vdash p, p' : \texttt{(a,b) t} \quad p \equiv_\Phi p' \quad n_1, \ldots, n_k \in \mathbb{N}}{\Gamma \vdash \mathit{par}\ [n_1, \ldots, n_k]\ p \equiv_\Phi \mathit{par}\ [n_1, \ldots, n_k]\ p'}$$

$$\frac{\Gamma \vdash p, p' : \texttt{(a,b) t} \quad p \equiv_\Phi p' \quad n_1, \ldots, n_k \in \mathbb{N}}{\Gamma \vdash \mathit{extend}\ [n_1, \ldots, n_k]\ p \equiv_\Phi \mathit{extend}\ [n_1, \ldots, n_k]\ p'}$$

$$\frac{\Gamma \vdash q, q' : \texttt{(a*a,a) t} \quad q \equiv_\Phi q' \quad n_1, \ldots, n_k \in \mathbb{N}}{\Gamma \vdash \mathit{reduce}\ [n_1, \ldots, n_k]\ q \equiv_\Phi \mathit{reduce}\ [n_1, \ldots, n_k]\ q'}$$

Fig. 4. Performance equivalence for AMAI terms.

The span of expressions that can be constructed in AMAI is very large, but many of them abstract operations that exhibit similar traits when carefully profiled and analyzed. Performance equivalence is vital for guaranteeing compositionality: equivalent terms share the same performance semantics, and thus can share the same empirical analysis and calibration. This motivates the addition of a system of denotational equivalence of AMAI terms based on traceable performance, not on executional equality. Two atomic terms sharing the same label shall be considered performance-equivalent, regardless of their encapsulated functions. Sequential and parallel computations are performance-equivalent when their terms are; for shaped combinators, we ask that shapes match too. The full system appears in Figure 4.

IV. Symbolic cost semantics

Describing AMAI programs consistently in terms of performance and resource consumption requires establishing a system of cost semantics. We must define unambiguously how to price the building blocks, and how to obtain the cost of a composite expression as a transformation of the cost of their components. We do so explicitly, by introducing a functorial transformation $\mathbb{M}$ that determines univocally the semantics of each combinator. Many other choices of $\mathbb{M}$ could have been made with reasonable motivations. Our pick is founded on generality of use and descriptional closeness to modern high-performance computing techniques. Our bridging model is an implementation of the AMAI interface that abstracts computations into time and memory consumption using four real values: total execution time, input size, output size and per-device memory. Each AMAI combinator is directly associated with an expression in terms of the subcomponents involved. Within the model we use constants for values that are measured and set once, and structural parameters for values

that can be changed depending on the architecture that runs the computation and the execution environment.

$$\mathbb{M} : \textbf{AMAI} \to \mathbb{R}^4 \tag{1}$$
$$\mathbb{M} = [\![\cdot]\!]_t \times [\![\cdot]\!]_i \times [\![\cdot]\!]_o \times [\![\cdot]\!]_m \tag{2}$$

Atomic operations are completely tuned by the user, since their behaviour is unknown by hypothesis. The empirical constants that determine them are set externally. We use bookkeeping functions ($\gamma_\square$) that scan the environment looking for values with a certain key.

$$[\![\, \textit{atom}\ s\ f \,]\!]_t = \gamma_t(s) \tag{3}$$
$$[\![\, \textit{atom}\ s\ f \,]\!]_i = \gamma_i(s) \tag{4}$$
$$[\![\, \textit{atom}\ s\ f \,]\!]_o = \gamma_o(s) \tag{5}$$
$$[\![\, \textit{atom}\ s\ f \,]\!]_m = \gamma_m(s) \tag{6}$$

For a sequence of operations, the total computation latency is the sum of latencies, while the needed memory per device is calculated as the maximum of the needed memories. Concurrent computations are dual: they finish at their maximum total latency, while requiring the sum of needed memory sizes. For neither of them do we introduce synchronization constants: whether there is synchronization or not, we leave the task of explicitly describing it to the user. This is convenient for modelling deep-learning programs, since the points at which the synchronization overhead is notable are memory transfers and communication blocks, which are already explicit. As for input and output sizes, we deduce them from their typing rules.

$$[\![\, p \ggg q \,]\!]_t = [\![\, p \,]\!]_t + [\![\, q \,]\!]_t \tag{7}$$
$$[\![\, p \ggg q \,]\!]_i = [\![\, p \,]\!]_i \tag{8}$$
$$[\![\, p \ggg q \,]\!]_o = [\![\, q \,]\!]_o \tag{9}$$
$$[\![\, p \ggg q \,]\!]_m = [\![\, p \,]\!]_m \vee [\![\, q \,]\!]_m \tag{10}$$

$$[\![\, p \,|||\, q \,]\!]_t = [\![\, p \,]\!]_t \vee [\![\, q \,]\!]_t \tag{11}$$
$$[\![\, p \,|||\, q \,]\!]_i = [\![\, p \,]\!]_i + [\![\, q \,]\!]_i \tag{12}$$
$$[\![\, p \,|||\, q \,]\!]_o = [\![\, p \,]\!]_o + [\![\, q \,]\!]_o \tag{13}$$
$$[\![\, p \,|||\, q \,]\!]_m = [\![\, p \,]\!]_m + [\![\, q \,]\!]_m \tag{14}$$

The three shaped combinators are modelled by considering a number of parallel workers P as a structural parameter, and adding constants for communication latency (α) and communication bandwidth (β), which is a well-established technique in the literature [25], [26]. These three values are set externally, and thus we identify $P = \gamma_P(p)$, $\alpha = \gamma_\alpha(p)$, $\beta = \gamma_\beta(p)$ to simplify the notation throughout the rest of this section. We justify the use of terms as keys for the environment using performance equality as the comparison function, since two performance-equivalent terms can share the same constants.

The cost expressions of *par* are deduced from the same logic as basic concurrent computations, since it represents another (shaped) concurrent operation. Now the total parallel work is divided equally among each worker, each of which needs to perform its tasks sequentially. This per-device definition of costs is consistent with the behaviour of distributed ML workloads. For instance, a chain of maps such as *par* $[\bar{n}]\ p \ggg$ *par* $[\bar{n}]\ q$ keeps data distributed and adds no latency cost beyond latencies of p and q.

$$[\![\, \textit{par}\ [n_1, ..., n_k]\ p \,]\!]_t = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_t \tag{15}$$
$$[\![\, \textit{par}\ [n_1, ..., n_k]\ p \,]\!]_i = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_i \tag{16}$$
$$[\![\, \textit{par}\ [n_1, ..., n_k]\ p \,]\!]_o = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_o \tag{17}$$
$$[\![\, \textit{par}\ [n_1, ..., n_k]\ p \,]\!]_m = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_m \tag{18}$$

The *extend* combinator is similar to *par* in terms of parallel work. To model its cost, it is necessary to include the communication overhead of distributing the input across workers. We choose to use the cost expression that corresponds to a tree-based data broadcast of the input size. The complete cost is equivalent to the sequential composition of the broadcast and the parallel computation.

$$C_{ext}(p) = log_2(P) \cdot (\alpha + \beta \cdot [\![\, p \,]\!]_i) \tag{19}$$
$$[\![\, \textit{extend}\ [n_1, ..., n_k]\ p \,]\!]_t = C_{ext}(p) + \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_t \tag{20}$$
$$[\![\, \textit{extend}\ [n_1, ..., n_k]\ p \,]\!]_i = [\![\, p \,]\!]_i \tag{21}$$
$$[\![\, \textit{extend}\ [n_1, ..., n_k]\ p \,]\!]_o = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_o \tag{22}$$
$$[\![\, \textit{extend}\ [n_1, ..., n_k]\ p \,]\!]_m = [\![\, p \,]\!]_i \vee \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, p \,]\!]_m \tag{23}$$

We model the *reduce* combinator as a distributed tree-based reduction, following the design choices made for *extend*. Again, this approach involves data communication, for which the latency also includes the cost of the reduction at each step. The cost semantics are coherent with the sequential version in which there is only one worker ($P = 1$), as the work share becomes the total amount of work and the communication costs vanish. The input equation transforms a fixed two-elements input size into a tensor input size.

$$C_{red}(q) = log_2(P) \cdot ([\![\, q \,]\!]_t + \alpha + \beta \cdot [\![\, q \,]\!]_o) \tag{24}$$
$$[\![\, \textit{reduce}\ [n_1, ..., n_k]\ q \,]\!]_t = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, q \,]\!]_t + C_{red}(q) \tag{25}$$
$$[\![\, \textit{reduce}\ [n_1, ..., n_k]\ q \,]\!]_i = \frac{\prod_{j=1}^{k} n_j}{P} \cdot \frac{1}{2} \cdot [\![\, q \,]\!]_i \tag{26}$$
$$[\![\, \textit{reduce}\ [n_1, ..., n_k]\ q \,]\!]_o = [\![\, q \,]\!]_o \tag{27}$$
$$[\![\, \textit{reduce}\ [n_1, ..., n_k]\ q \,]\!]_m = \frac{\prod_{j=1}^{k} n_j}{P} \cdot [\![\, q \,]\!]_m \vee [\![\, q \,]\!]_o \tag{28}$$

With *extend* and *reduce* it is possible to model composite collective communication directives and keep costs sound. Consider the *AllReduce* collective: it can be modelled as $allreduce\ n = \textit{reduce}\ [n]\ add \ggg \textit{extend}\ [n]\ id$. Our semantics translate the communication latency as $2 \cdot \log_2(P) \cdot (\alpha + \beta \cdot n) + \log_2(P) \cdot [\![add]\!]_t$, which is precisely the cost reported in the literature for a tree-based *AllReduce* [26], [29]. Techniques like tensor parallelism can be modelled by setting the P parameter of the extend combinator in the linear and convolutional layers (Figure 3) to the tensor-parallelism degree. The resulting cost expression matches the tensor-parallel communication formulas of Megatron-LM [7].

A. Case study: a distributed AI workload

As an example of how our abstract cost semantics map to concrete examples of distributed deep-learning workloads, we illustrate here the description of the design of a basic transformer-based decoder model's inference workload [30]. We show the global view of the model architecture, as a sequence of layers. In order to ease the reading of the script, we detail the model down to the sub-layer component level and assume the smaller blocks are already described. Some hyperparameters of the model, such as the number of attention heads, the per-head dimension or the compression/projection dimensions, are also omitted to simplify our figures.

```
let attention = triple
                >>> (linearQ ||| linearK ||| linearV)
                >>> core_att
                >>> linearO

let ffn = dup
          >>> ((linearW1 >>> vsilu) ||| linearW2)
          >>> vmul
          >>> linearW3

let layer = dup >>> ((norm >>> attention) ||| id) >>> add
        >>> dup >>> ((norm >>> ffn      ) ||| id) >>> add

let l = 24  (* Number of layers *)
let stack = let layers = List.init l (fun _ -> layer) in
            List.fold_left (>>>) id layers

let model = embed_in >>> stack >>> embed_out

let b = 1  (* Batch size *)
let dp = 4 (* Data Parallel degree *)
let batched = par [b] model
let full = par [dp] batched >>> reduce [dp] cat
```

Fig. 5. The architecture scheme of a transformer-based decoder model.

This description evaluates to a cost model whose description is below. We use the symbols B and DP for batch size and data parallelism degree, respectively. The first two equations are those immediately extracted from the tool. Parameters P_{DP} and P_{cat} correspond to the same value, the degree of data parallelism. Parameter P_B corresponds to the number of input elements that a worker can compute in parallel, a machine-specific value. Concatenating tensors requires virtually no additional time or memory if we disregard memory allocation and fragmentation, and its output size in this case is that of the own model. The second set of equations is derived after setting values and simplifying, and resemble the standard formulation for model inference in many analytical models [7], [31]. However, we stress that these equations do not hold for model training, since its computational description is significantly different.

$$[\![full]\!]_t = \frac{DP}{P_{DP}} \cdot \frac{B}{P_B} \cdot [\![model]\!]_t + \frac{DP}{P_{cat}} \cdot [\![cat]\!]_t \quad (29)$$

$$+\ log_2(P_{cat}) \cdot ([\![cat]\!]_t + \alpha + \beta \cdot [\![cat]\!]_o) \quad (30)$$

$$[\![full]\!]_m = \frac{DP}{P_{DP}} \cdot \frac{B}{P_B} \cdot [\![model]\!]_m \quad (31)$$

$$\vee \frac{DP}{P_{cat}} \cdot [\![cat]\!]_m \vee [\![cat]\!]_o \quad (32)$$

$$[\![full]\!]_t = \frac{B}{P_B} \cdot [\![model]\!]_t \quad (33)$$

$$+\ log_2(DP) \cdot (\alpha + \beta \cdot [\![model]\!]_o) \quad (34)$$

$$[\![full]\!]_m = \frac{B}{P_B} \cdot [\![model]\!]_m \vee [\![model]\!]_o \quad (35)$$

V. Evaluation

In order to validate our contribution, we show that it is as effective at the task of predicting the resource requirements of AI workloads as the brute-force approach, while requiring fewer compute resources. We will test our approach against the classic exhaustive profiling method as a reference for measurements and expected behaviour, focusing on prediction accuracy for both time and memory.

A. Testbed and methodology

a) Hardware environment: All experiments run on Ascend 910C nodes, each holding 8 Ascend 910C accelerators with two dies with 64 GiB of high-bandwidth memory per device, connected by a high-bandwidth intra-node fabric and a slower inter-node network. Each die is exposed as a separate device, so configurations span one to two nodes, that is, 1 to 32 devices; the largest of them cross the node boundary and exercise both levels of the interconnect.

b) Target workload: We train the Qwen3.5 0.8B model, from the Qwen3.5 dense family [32], [33]. We train using HyperParallel [21] on torch 2.7.1 and torch-npu 2.7.1 [6]. Training runs in mixed precision with *bfloat16* parameters, *float32* gradient reduction, and *bfloat16* outputs, under AdamW with a cosine schedule. We use a synthetic text dataset of uniform sequence length, and adjust the sequence length value depending on the experiments performed. We designed the complete training of the model in AMAI, namely the forward pass, backward pass, the gradient reduction and the weight update with AdamW. While a scheme of the inference computations is depicted in Section IV-A, the full description involves saving partial activations, reducing gradients with communication collectives and grouping the parameter updates. The final cost model extracted with our tool includes more than 200 variables, and can be automatically generated with the material provided as part of this contribution.

c) Experimental setup: We propose two sets of experiments, in order to test the precision of the system as a tool for performance prediction. In the first set, we set sequence length to 4096 and vary the batch size (B) in the range $[1, 2, 4, 8]$ to validate our calibration of computation constants against real measurements using one device. In the second set, we set sequence length to 1024 and scale out the computation using data parallelism: we choose DP degrees in the range $[1, 2, 4, 8, 16, 32]$.

d) Metrics: Two metrics are measured and reported: the step time in seconds and the per-device peak memory in gigabytes (GiB). The memory value reported corresponds to the peak memory allocated by the system to run the target workload. Since this value is deterministic across launches, we measure it once. Each training configuration runs for 20 steps. We report median time values over the 20 steps; the per-step standard deviation stabilizes afterwards at 5 ms.

e) Calibration: The constants of the derived cost model were fitted by profiling individual implementations of atomic blocks and communication constants, and manually setting the structural worker parameters. Most of the primitives (*add*, *mul*, *dup*) are cheap to profile: a short run suffices to fit their throughput and memory constants. Structural worker parameters were manually set, matching the structure of the underlying hardware and accounting for the vector and cube units used for SIMD operations. For instance, the number of workers associated with the batching of the input always matches the batch number, since even inside one accelerator, these computations can be performed in parallel. We ran a reduced configuration of Qwen3.5 with one layer, on one device, to profile all computation constants for the full model. The latency and bandwidth constants are already given by the hardware specifications of the links between dies, devices and nodes. However, we checked that links run close to their peak at the message sizes involved.

B. Results

1) Intra-device: Our first set of experiments confirms that our empirical inference of constants for machine parameters is valid. The difference between measured and predicted metrics is minimal: an average relative error of 1.92 percent for step time and 2.69 percent for predicted memory, as seen in Figure 6. These differences can be explained by underlying optimizations for memory allocation and for the latencies from low-level routines outside of the control of the programmer, which are hidden behind the hardware library controller and cannot be modelled.

To emphasize the contrast between the classic method and ours, the full model has 24 layers, while we only needed to profile one of them to obtain the empirical values of computation constants. The time to run the calibration profiling round for constants was 0.0762 seconds in median. Assuming that the time values we reported were constant across all 20 measurements per configuration profiling, the fraction of machine usage we need in comparison with the classic profiling method is $f = \frac{0.0762}{1.705+1.855+1.94+2.355} \approx 0.97$ percent. The

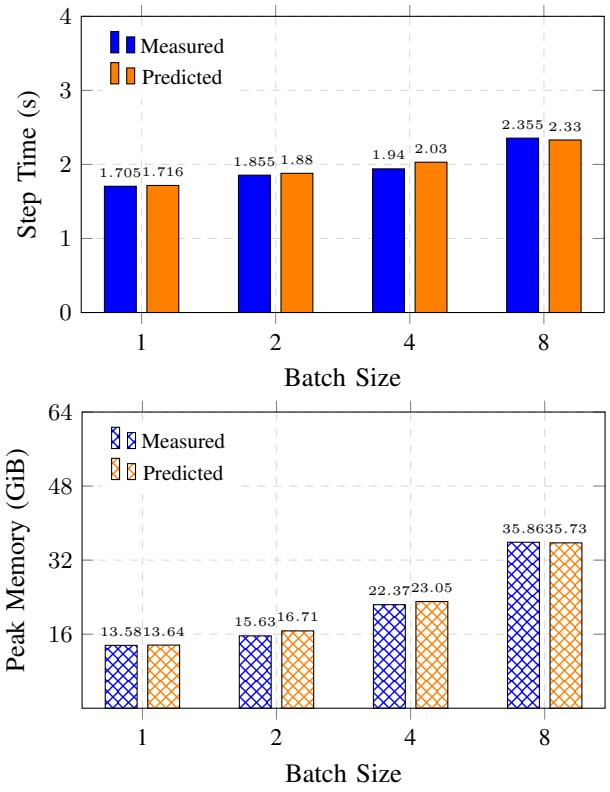


Fig. 6. Prediction of time and memory for the first set of experiments, scaling of batch size (B)

fraction would have been much lower had we included more experiments in our setup.

2) Inter-device: The data parallel scheme incurs nontrivial communication overhead during backpropagation, where different devices share the gradients with respect to the model parameters. Hence, correctly setting latency and bandwidth constants for the communication links and buses involved is crucial: between two dies within the same Ascend 910C, bandwidth is much higher than between dies in different devices. This distinction can be described by setting different values in the corresponding latency and bandwidth parameters. The outcome is shown in Figure 7: we obtain 1.15 percent relative error predicting time. The memory prediction has a 2.22 percent relative error. The only visible deviation occurs at DP = 1, where the model overestimates peak memory by 2.6 GiB. This is caused by a framework-specific memory fragmentation optimization that appears where no gradient reduction takes place.

3) Discussion and limitations: Our evaluation focuses on a restricted set of parallelism techniques, over a 0.8B-parameter dense model on up to two nodes. The cost model requires a one-off calibration per hardware platform: constants fitted on Ascend 910C do not transfer to other accelerators. The description of the program does not model the low-level kernels, overlooking the potential impact of the cache hierarchy. The collective communications modelled are assumed as tree-based (as implemented in HyperParallel, the framework we

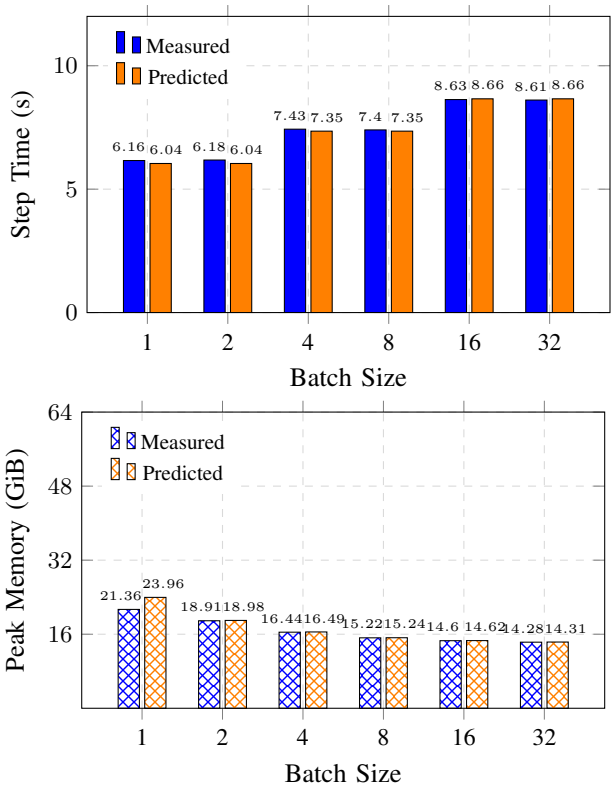


Fig. 7. Prediction of time and memory for the second set of experiments, scaling data parallelism (DP)

use [21]) instead of following ring-based or halving-doubling implementations, whose cost has a different expression. A more thorough definition could consider all of them depending on message sizes. Overlap of computation and communication requires carefully describing computations and is not captured automatically within the semantics of $\ggg$ and $|||$. Finally, we model peak allocated memory and deliberately exclude allocator fragmentation and other low-level runtime effects, which bounds the attainable accuracy, as reflected in the residual 2–3 percent error.

VI. Related work

Existing approaches differ in the representation they analyze and in the cost paid per configuration evaluated; following the broad taxonomy of [34], we review them in order of increasing intensity of machine usage.

a) Analytical performance models: A first family of approaches concerns the estimation of execution time, memory footprint, or throughput of distributed deep-learning through closed-form analytical expressions. Paleo [31] explores parallel training configurations by deriving costs from neural network specifications; AMPeD [35] and Calculon [36] parameterize end-to-end models of distributed training by model dimensions, parallelization strategy, and hardware characteristics. Training frameworks like Megatron-LM [7] or Alpa [37] include similar per-dimension formulas to guide their own configuration. These methods are fast and interpretable, but hard-code fixed equations: studying a new architecture or a new composition of parallel techniques not covered before requires re-deriving the cost model by hand. AMAI targets this lack of compositionality. The cost model is not written per architecture but derived from a program description, so the same semantics can apply to arbitrary compositions of operators and parallel patterns.

b) Languages with cost semantics: Methodologically, our work belongs to the lineage of array and data-parallel languages with explicit cost semantics. NESL [38], [39] pioneered the idea as a programming language whose programs can be mapped to execution time. Nowadays, users can choose from a selection of array and tensor programming languages (APL [40], Futhark [41], Halide [42] and Lift [43] among them) that generate highly efficient code. Algebras of arrays like MoA [44], [45] and algorithmic skeletons [46] incorporate parallelism at a higher level, where it can be combined with symbolic performance models, and attach cost models to reusable parallel patterns in a similar way to our contribution. AMAI differs from these approaches in that it offers no execution semantics; our DSL is meant to be a purely representational language. The trade-off allows placeholders and empirical constants to coexist with structural descriptions, and makes the method fully operable and applicable to systems with arbitrary execution models.

c) Simulators: Simulators ingest a workload representation and a system configuration, and simulate the execution of operator graphs and network communications to compute execution time, communication overhead and memory behaviour. There is a wide variety, from ONNXim [47] and Accel-Sim [48] which offer single-device cycle-level detail, to cluster simulators for distributed training like ASTRA-SIM [49], SimAI [50], and Proteus [51]. Their precision is higher than that of analytical models, since they can capture specific compute–communication overlap and allocator behaviour that lighter approaches (including our contribution) may abstract away [34]. Their prediction errors for data-parallelism workloads reach their lowest at around 3 percent [50], [51]. Nevertheless, every configuration of interest must be simulated anew, and the machinery is substantially heavier. AMAI ingests a representation of the computational workload, but it outputs symbolic parameterized cost equations, rather than final values calculated through an algorithmic replication of a concrete execution.

d) Profile-driven analyzers: A more accurate and profiling-intensive variant is the set of analyzers that parse execution profiles and extrapolate their details to predict other configurations [52]–[54]. As for this contribution, AMAI achieves comparable accuracy to these analyzers in the class of experiments performed: Lumos, the most recent work, reports 1.77 percent average error on predictions for similar data-parallelism strategies. The fundamental design difference between AMAI and these tools lies in the modeling approach: we push the users to design and specify their programs by themselves, and then we give them tools to extract their cost models. As such, the input of AMAI is a high-level descrip-

tion, rather than a low-level trace file that is processed accordingly to predefined models of performance. Furthermore, analyzers are strongly coupled to the frameworks that emit them, while AMAI is bounded by the language's expressivity rather than what specific traces can capture.

VII. Conclusion

We presented AMAI, a framework for performance analysis of AI workloads, consisting of a functional domain-specific language for rapid description of deep learning computations, together with a cost modelling framework and a functorial translation that maps its programs to symbolic resource requirements. The system allows users to encode heterogeneous representations of computations and abstracts machine-specific details into numerical parameters, providing high flexibility to express a broad diversity of workloads and environments. Computational descriptions are mapped automatically into the resources required for execution. These costs are sound, in the sense that complex collectives studied in the literature can be expressed in our language and are given the same cost. Furthermore, because the semantics are compositional, the cost of a program follows from the cost of its parts: reusability is increased over time. We validated the approach by training a Qwen3.5 0.8B model on Ascend 910C accelerators. After calibrating the constants by profiling individual operations, predictions of step time and per-device peak memory differed from measurements by 1.92 and 2.69 percent on average when scaling the batch size, and by 1.15 and 2.22 percent on average when scaling out with data parallelism. These results illustrate the three properties targeted in Section I: compositionality, since the cost of the full training step is derived from that of its components; operability, since profiled constants and structural placeholders coexist in a single description; and parametricity, since the same symbolic models are re-evaluated across all batch-size and device-count configurations without re-derivation.

As future steps, we plan to describe more advanced techniques for parallelism in deep learning computations using AMAI and compare with existing work. Tensor and expert parallelism have already been covered at length in related work [7], [34], [36]. Pipeline parallelism, another of our targets, has many possible implementations depending on the training scheme. Given the complexity of its description, it has also been left for future work.

Acknowledgment

This work was partially supported by the National Key R&D Program of China (Grant No.2023ZD0120400).

References

[1] N. Maslej *et al.*, "Artificial intelligence index report 2025," 2025. Available: https://arxiv.org/abs/2504.07139

[2] A. Hussain, U. E. Farwa, S. Ali, and H.-C. Kim, "The rise of foundation models: Opportunities, technology, applications, challenges, recent trends, and future directions," *Appl. Syst. Innov.*, vol. 9, no. 2, 2026, Art. no. 35. doi:10.3390/asi9020035

[3] F. Lin, "Vision language models: A survey of 26k papers," 2025. Available: https://arxiv.org/abs/2510.09586

[4] C. E. Leiserson *et al.*, "There's plenty of room at the top: What will drive computer performance after Moore's law?" *Science*, vol. 368, no. 6495, p. eaam9744, 2020. doi:10.1126/science.aam9744

[5] T. Ben-Nun and T. Hoefler, "Demystifying parallel and distributed deep learning: An in-depth concurrency analysis," *ACM Comput. Surv.*, vol. 52, no. 4, Aug. 2019, Art. no. 65. doi:10.1145/3320060

[6] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33st Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates Inc., 2019. doi:10.48550/arXiv.1912.01703

[7] D. Narayanan *et al.*, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. doi:10.1145/3458817.3476209

[8] W. Liang *et al.*, "TorchTitan: One-stop PyTorch native solution for production ready LLM pretraining," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 92 161–92 181.

[9] Huawei Technologies Co., Ltd., "Huawei MindSpore AI development framework," in *Artificial Intelligence Technology*. Springer, 2022, pp. 137–162.

[10] S. Deng *et al.*, "Cloud-native computing: A survey from the perspective of services," *Proc. IEEE*, vol. 112, no. 1, pp. 12–46, 2024. doi:10.1109/JPROC.2024.3353855

[11] J. Gusak *et al.*, "Survey on efficient training of large neural networks," in *Proc. 31st Int. Joint Conf. Artif. Intell. (IJCAI-22)*, L. De Raedt, Ed., Jul. 2022, pp. 5494–5501, Survey Track. doi:10.24963/ijcai.2022/769

[12] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009. doi:10.1145/1498765.1498785

[13] L. G. Valiant, "A bridging model for parallel computation," *Commun. ACM*, vol. 33, no. 8, pp. 103–111, Aug. 1990. doi:10.1145/79173.79181

[14] D. Culler *et al.*, "LogP: Towards a realistic model of parallel computation," in *Proc. 4th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, ser. PPOPP '93. New York, NY, USA: Association for Computing Machinery, 1993, pp. 1—-12. doi:10.1145/155332.155333

[15] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986. doi:10.1038/323533a0

[16] X. Hao, G. Zhang, and S. Ma, "Deep learning," *Int. J. Semant. Comput.*, vol. 10, no. 3, pp. 417–439, Sep. 2016. doi:10.1142/S1793351X16500045

[17] S. Ruder, "An overview of gradient descent optimization algorithms," 2017. Available: https://arxiv.org/abs/1609.04747 (Accessed 09-2003).

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, Y. Bengio and Y. LeCun, Eds., San Diego, CA, USA, May 2015. Available: http://arxiv.org/abs/1412.6980

[19] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Oper. Syst. Design Implement. (OSDI 16)*, Savannah, GA, Nov. 2016, pp. 265–283. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[20] J. Bradbury *et al.*, "JAX: Composable transformations of Python+NumPy programs," 2018, Version 0.3.13. Available: http://github.com/jax-ml/jax

[21] X. Zhang *et al.*, "HyperParallel: A supernode-affinity AI framework," 2026, arXiv:2603.03731.

[22] A. Alexandrov, M. F. Ionescu, K. E. Schauser, and C. Scheiman, "LogGP: Incorporating long messages into the LogP model for parallel computation," *J. Parallel Distrib. Comput.*, vol. 44, no. 1, pp. 71–79, 1997. doi:10.1006/jpdc.1997.1346

[23] L. G. Valiant, "A bridging model for multi-core computing," vol. 77, no. 1. USA: Academic Press, Inc., Jan. 2011, pp. 154—-166. doi:10.1016/j.jcss.2010.06.012

[24] C. Li and G. Hains, "Sgl: towards a bridging model for heterogeneous hierarchical platforms," *Int. J. High Perform. Comput. Netw.*, vol. 7, no. 2, pp. 139—-151, Apr. 2012. doi:10.1504/IJHPCN.2012.046371

[25] R. W. Hockney, "The communication challenge for MPP: Intel Paragon and Meiko CS-2," *Parallel Comput.*, vol. 20, no. 3, pp. 389–398, 1994. doi:10.1016/S0167-8191(06)80021-9

[26] E. Chan, M. Heimlich, A. Purkayastha, and R. van de Geijn, "Collective communication: Theory, practice, and experience," *Concurrency Comput., Pract. Exper.*, vol. 19, no. 13, pp. 1749–1783, 2007. doi:10.1002/cpe.1206

[27] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.

[28] J. Hughes, "Generalising monads to arrows," *Sci. Comput. Program.*, vol. 37, no. 1, pp. 67–111, 2000. doi:10.1016/S0167-6423(99)00023-4

[29] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, 2005. doi:10.1177/1094342005051521

[30] A. Vaswani *et al.*, "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*", ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 6000—6010.

[31] H. Qi, E. R. Sparks, and A. Talwalkar, "Paleo: A performance model for deep neural networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017. Available: https://openreview.net/forum?id=SyVVJ85lg

[32] Qwen Team, "Qwen3.5: Towards native multimodal agents," Feb. 2026. Available: https://qwen.ai/blog?id=qwen3.5 (Accessed 2026-09-25).

[33] A. Yang *et al.*, "Qwen3 technical report," 2025, arXiv:2505.09388.

[34] Z. Guo *et al.*, "A survey on performance modeling and prediction for distributed DNN training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 12, pp. 2463–2478, Dec. 2024. doi:10.1109/TPDS.2024.3476390

[35] D. Moolchandani, J. Kundu, F. Ruelens, P. Vrancx, T. Evenblij, and M. Perumkunnil, "AMPeD: An analytical model for performance in distributed training of transformers," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2023, pp. 306–315. doi:10.1109/ISPASS57527.2023.00037

[36] M. Isaev, N. Mcdonald, L. Dennison, and R. Vuduc, "Calculon: a methodology and tool for high-level co-design of systems and large language models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. doi:10.1145/3581784.3607102

[37] L. Zheng *et al.*, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," 2022, arXiv:2201.12023.

[38] G. E. Blelloch, "Programming parallel algorithms," *Commun. ACM*, vol. 39, no. 3, pp. 85–97, Mar. 1996. doi:10.1145/227234.227246

[39] G. E. Blelloch and J. Greiner, "A provable time and space efficient implementation of nesl," in *Proc. 1st ACM SIGPLAN Int. Conf. Funct. Program.*, ser. ICFP '96. New York, NY, USA: Association for Computing Machinery, 1996, pp. 213—-225. doi:10.1145/232627.232650

[40] K. E. Iverson, *A programming language*. USA: John Wiley & Sons, Inc., 1962.

[41] T. Henriksen, N. G. W. Serup, M. Elsman, F. Henglein, and C. E. Oancea, "Futhark: Purely functional GPU-programming with nested parallelism and in-place array updates," in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, Barcelona, Spain, 2017, pp. 556–571. doi:10.1145/3062341.3062354

[42] J. Ragan-Kelley *et al.*, "Halide: Decoupling algorithms from schedules for high-performance image processing," *Commun. ACM*, vol. 61, no. 1, pp. 106–115, Dec. 2017. doi:10.1145/3150211

[43] M. Steuwer, T. Remmelg, and C. Dubach, "Lift: A functional data-parallel IR for high-performance GPU code generation," in *Proc. Int. Symp. Code Gener. Optim. (CGO)*, Austin, USA, 2017, pp. 74–85.

[44] G. Hains and L. M. R. Mullin, "Parallel functional programming with arrays," *Comput. J.*, vol. 36, no. 3, pp. 238–245, Jan. 1993. doi:10.1093/comjnl/36.3.238

[45] G. Hains and L. Mullin, "New mathematics for computer performance: Array algebra and cost functions," *Mathematics*, vol. 14, no. 9, 2026, Art. no. 1479. doi:10.3390/math14091479

[46] M. Cole, "Bringing skeletons out of the closet: A pragmatic manifesto for skeletal parallel programming," *Parallel Comput.*, vol. 30, no. 3, pp. 389–406, 2004. doi:10.1016/j.parco.2003.12.002

[47] H. Ham *et al.*, "ONNXim: A fast, cycle-level multi-core NPU simulator," *IEEE Comput. Archit. Lett.*, vol. 23, no. 2, pp. 219–222, 2024. doi:10.1109/LCA.2024.3484648

[48] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 473–486. doi:10.1109/ISCA45697.2020.00047

[49] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling SW/HW co-design exploration for distributed DL training platforms," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2020, pp. 81–92. doi:10.1109/ISPASS48437.2020.00018

[50] X. Wang *et al.*, "SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision," in *Proc. 22nd USENIX Symp. Netw. Syst. Design Implement. (NSDI 25)*, Philadelphia, PA, Apr. 2025, pp. 541–558. Available: https://www.usenix.org/conference/nsdi25/presentation/wang-xizheng-simai

[51] J. Duan *et al.*, "Proteus: Simulating the performance of distributed DNN training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 10, pp. 1867–1878, 2024. doi:10.1109/TPDS.2024.3443255

[52] H. Zhu, A. Phanishayee, and G. Pekhimenko, "Daydream: Accurately estimating the efficacy of optimizations for DNN training," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC 20)*, Jul. 2020, pp. 337–352. Available: https://www.usenix.org/conference/atc20/presentation/zhu-hongyu

[53] G. X. Yu, Y. Gao, P. Golikov, and G. Pekhimenko, "Habitat: A runtime-based computational performance predictor for deep neural network training," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC 21)*, Jul. 2021, pp. 503–521. Available: https://www.usenix.org/conference/atc21/presentation/yu

[54] M. Liang, H. T. Kassa, W. Fu, B. Coutinho, L. Feng, and C. Delimitrou, "Lumos: Efficient performance modeling and estimation for large-scale LLM training," in *Proc. Mach. Learn. Syst.*, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7, 2025. Available: https://proceedings.mlsys.org/paper_files/paper/2025/file/a66caa1703fe34705a4368c3014c1966-Paper-Conference.pdf

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

C_1 A domain-specific language for specifying descriptions of tensor computations.

C_2 A system of cost semantics that transform expressions in the previous DSL into symbolic models of time and per-device memory.

B. Computational Artifacts

A_1 Software system that integrates C_1 and C_2 into a tool for performance modeling and cost analysis.
Source: https://github.com/TheNuber/AMAI.
DOI: https://doi.org/10.5281/zenodo.22895946.

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2	Figures 1-7

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

The following artifact corresponds to the software suite used to generate models and translate them to software costs. It contains a small library that includes the domain-specific language implementation, and some test folders with the material needed to carry out the modeling process.

Expected Results

Expected result for each of the tests included with the suite is the generation of tunable parameterized cost models of performance, that can be modified and processed to return numerical end-to-end costs. After a careful tuning of the parameters, it is possible to obtain results similar to those highlighted in the main content sections.

Expected Reproduction Time (in Minutes)

- Setup time: 2 – 150 minutes (clone repo + install software).
- Execution time: 1 – 3 minutes.
- Analysis time: 1 – 5 minutes to read the model and generate results.

Artifact Setup (incl. Inputs)

Hardware: Commodity Linux, Windows or Unix machines, preferably with modern CPU and RAM capabilities, are enough for the reproduction of this artifact. No GPU is needed.

Software: The following packages are needed to use this artifact, assuming a Linux-based operative system:

1) OCaml 5.3.0. URL: https://ocaml.org/install
2) Opam 2.3.0. URL: https://opam.ocaml.org/
3) Dune 3.23.1. URL: https://github.com/ocaml/dune

Datasets / Inputs: The performance descriptions and specifications from which the cost models are generated are included in separate test folders within the software package of this artifact. The cost models can be generated automatically with provided scripts.

Installation and Deployment: Installation only requires cloning the repository available through the source provided: `git clone https://github.com/TheNuber/AMAI` should work just fine. It is needed to download Opam as a package manager, and install OCaml and Dune with it: `opam install ocaml dune`. To build the project, run `dune build` from the root folder.

Artifact Execution

The workflow for each test is the following chain:

- `cd TESTFOLDER`: Go to the test folder of choice
- `dune exec TESTFOLDER generate`: generate a CSV file `env_template.csv` with the parameters of the model that the test describes, as well as a text file `model.txt` with a visual representation of the cost model expressions.
- `dune exec TESTFOLDER analyze`: read the CSV file and evaluate the model with the values included. The final values will be stored in a text file `final_values.txt`.

Artifact Analysis (incl. Outputs)

The first file to analyze is `model.txt`, since it contains the visual description of the generated model. The CSV file `env_template.csv` traces the variables involved, and acts as a key/value dictionary. The file `final_values.txt` will contain the final values of the execution, and without a proper assignment of variable values it is expected that it contains values such as `nan`.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

The installation of this artifact can be divided in two parts: the installation of the OCaml environment, and the installation of the software package corresponding to the artifact itself.

Installing the OCaml environment, including Opam and Dune, is straightforward from the website. A workflow could be summarized in the following, in Linux:

```
sudo apt install opam
opam install ocaml dune
```

Installing the software package requires cloning the repository, either via HTTPS or SSH, and building the source. For instance:

```
git clone https://github.com/TheNuber/AMAI
cd AMAI
dune build
```

Artifact Execution

The workflow needed to model the target workloads is encapsulated in the test folders, and great part of the work is summarized within the test files.

- `example_inference` describes informally the specification of a simple inference workload with a transformer-based model.
- `training_step_qwen35_08B` is the test on which we based the modeling and experiments that output the results shown in the manuscript.

We provide a way of automatically generating the cost model with `dune exec TESTFOLDER generate` and evaluating the cost with `dune exec TESTFOLDER analyze`. The reader must fill the values of the model *before* running the analysis, to obtain correct measurements. Most of them correspond to basic values (the assignment of $P = \gamma_P(\cdot)$ is almost always one, and the input and output sizes can be derived from the atoms that contain them), and others can be easily inferred.

Artifact Analysis (incl. Outputs)

The expected results are a textual representation of the cost model and a file that contains the final outputs. The final outputs depend on the values inputted in the CSV variable environment file.

In order to obtain values similar to those depicted in the figures of the manuscript, it is needed to run the analysis once per configuration of variable values. That is, if Figure X shows Y bars corresponding to Y different configurations, it is needed to run the analysis Y times with the correct parameter values to obtain the data. As shown in the manuscript, it is only needed to vary the parameters that correspond to batch size and data parallelism, thus reducing execution time to a few minutes.